Vulnerability Detection and Patch Creation

WEREVUL

Turan Emre Adıgüzel
Elif Duman
Umut Övünç Şenol
Merve Topuz
Zeynep Taşevli

CHALLENGES & RISKS

Problem Definition

Repository Scale

Large Java repositories are difficult to inspect manually.

Static Analysis Noise

Static analysis tools may generate many false positives.

AI Reliability

AI models may produce convincing but incorrect results.

Detection Verification

LLMs show false positive results time to time

Project Overview

What is the project about?

Core Goal

Develop an LLM-assisted framework to detect, localize, and propose fixes for security vulnerabilities in Java repositories.

Input

Java source code repository.

Analysis

Static analysis, file/function localization, and LLM-based vulnerability reasoning.

Output

Vulnerability reports containing CWE classification, affected code locations, explanations, and suggested patches.

USER GROUPS

Our Target Audience

WereVul is designed for teams that need an accessible, privacy-preserving solution to detect, understand, and repair vulnerabilities in Java projects.

Java Developers & Teams

Java developers and software development teams building scalable applications.

AppSec Engineers

Application security engineers focusing on risk mitigation and code auditing.

Open-Source Maintainers

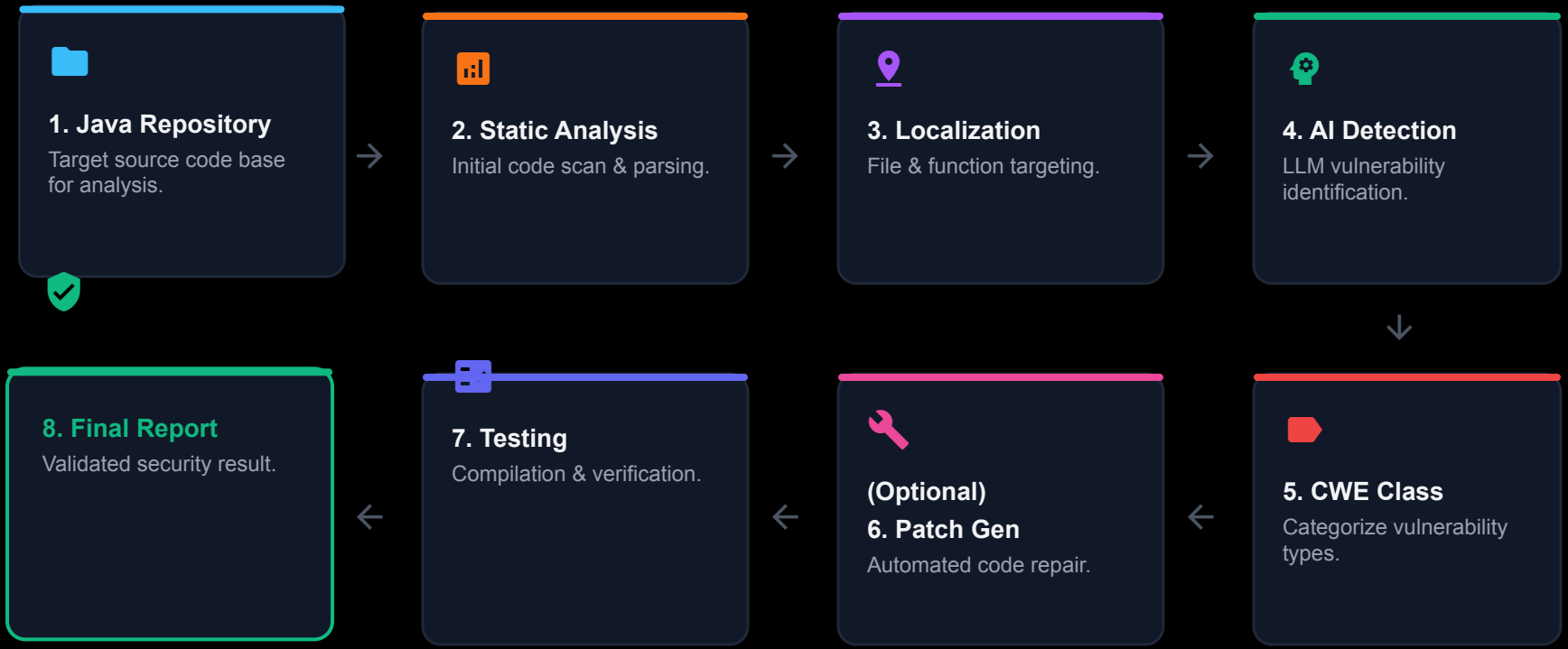
Open-source project maintainers protecting community ecosystems and libraries.

Growing Teams

Small and medium-sized teams without dedicated security experts.

SYSTEM ARCHITECTURE

How WereVul Works



CAPABILITIES

Key Features

- **Detects vulnerabilities in repositories** — Deep context scanning across project files
- **Identifies affected files, functions, and code lines** — Precise localization down to line level
- **Classifies findings using CWE categories** — Standardized weakness taxonomy
- **Supports locally running AI models for code privacy** — On-premise execution
- **Produces clear and actionable vulnerability reports** — Developer-friendly insights

Optional

- **Generates AI-assisted repair suggestions** — Automated patch proposal

PROJECT DELIVERABLES

Outcomes of Our Project

LLM-Supported Pipeline

Build an end-to-end automated pipeline leveraging multiple Large Language Models (LLMs) to detect software vulnerabilities effectively.

Research & Publication

Publish a research paper evaluating the efficiency and detection performance of various LLM tools in identifying vulnerabilities.

INNOVATION

What makes WereVul Innovative

- **Combines static analysis with local code language models** — Dual-approach vulnerability detection
- **Reduces large repositories before sending code to the AI model** — Optimized context windows
- **Keeps source code inside the local environment** — Complete data privacy & security
- **Does not accept an AI-generated patch without validation** — Strict patch verification
- **Uses Proof-of-Vulnerability and regression tests** — Automated safety checks

BENEFITS

Key Advantages of WereVul

- **Local & Privacy-First Execution** — Keeps source code secure inside the local environment
- **High-Precision Detection** — Rapidly identifies code vulnerabilities down to line level
- **Actionable CWE Insights** — Standardized vulnerability taxonomy for clear remediation
- **Efficient Repository Analysis** — Reduce the amount of code processed by LLMs through targeted localization

EVALUATION

Experimental Evaluation

Datasets: Evaluating WereVul using reproducible Java datasets including **Vul4J** and selected **Juliet Test Suite** cases.

- **Vulnerability detection accuracy** — Precision in identifying potential security risks
- **CWE classification accuracy** — Standardized weakness taxonomy mapping
- **File, function, and line localization accuracy** — Precise error tracing down to line level
- **Patch compilation success** — Ensuring generated patches compile cleanly
- **Security test and regression test success** — Guaranteeing zero broken builds or new bugs
- **Analysis time and resource consumption** — Measuring system efficiency and compute overhead